

Documento explicativo del proyecto PM2.5 Lima

Limpieza de datos, notebooks, modelos Lasso Regression y Decision Tree Regressor, variables usadas y comparacion de resultados.

1. Resumen del proyecto

El proyecto analiza datos horarios de contaminantes del aire en Lima Metropolitana. El objetivo practico es estimar la concentracion de PM2.5 usando variables ambientales, temporales y, para el arbol de decision, la estacion de monitoreo codificada.

- Dataset original: data/raw/datos_horarios_contaminacion_lima.xlsx.
- Variable objetivo: PM 2.5.
- Modelos implementados: Lasso Regression y Decision Tree Regressor.
- Dashboard: app Flask con metricas, graficos y formulario de prediccion.
- Modelos guardados: models/lasso_regression.pkl y models/decision_tree_regressor.pkl.

2. Estructura del proyecto

Carpeta / archivo	Funcion
app/	Dashboard Flask: rutas, HTML, CSS, JavaScript y predictor.
data/raw/	Dataset original sucio. Se conserva el nombre datos_horarios_contaminacion_lima.xlsx.
data/processed/	Datos generados por limpieza: datos_limpios.csv y datos_modelo.csv.
data/test/	Datos de prueba separados para evaluar los modelos.
notebooks/	Notebooks explicativos de limpieza, Lasso, Decision Tree y comparacion.
models/	Archivos .pkl generados con joblib. Cada .pkl guarda el pipeline completo del modelo.
scripts/	Codigo reproducible para limpiar, entrenar y evaluar.
reports/	Metricas, resultados en Excel, reglas del arbol e imagenes del dashboard.

3. Datos y variables

Las variables se toman del dataset original despues de limpiar duplicados, convertir contaminantes a numericos, imputar predictores faltantes y conservar filas donde PM 2.5 esta observado. No se predice contra valores inventados de la variable objetivo.

Variable	Tipo	Uso en el modelo
PM 10	Numerica	Predictora principal. Aparece como la primera decision del arbol.
SO2	Numerica	Predictora contaminante.
NO2	Numerica	Predictora contaminante.
O3	Numerica	Predictora contaminante.
CO	Numerica	Predictora contaminante.
ANO, MES, DIA, HORA	Numericas temporales	Representan contexto temporal; HORA destaca en el arbol.
ESTACION	Categorica/texto	No se usa en Lasso. En Decision Tree se convierte con OneHotEncoder.
PM 2.5	Numerica	Variable objetivo que se desea predecir.

4. Notebooks del proyecto

Notebook	Contenido principal
01_ Limpieza_ Datos.ipynb	Notebook de limpieza trabajado: inspeccion, nulos, conversiones, visualizaciones y preparacion inicial del dataset.
04_ Lasso_ Regression.ipynb	Implementa Lasso con variables numericas. Explica que no se usa ESTACION porque es texto.
05_ Decision_ Tree_ Regressor.ipynb	Implementa Decision Tree con variables numericas y ESTACION codificada mediante OneHotEncoder.
06_ Comparacion_ Modelos.ipynb	Compara metricas y ayuda a interpretar que modelo funciona mejor.

El notebook de regresion logistica se uso solo como referencia de estilo metodologico: importar librerias, cargar datos, separar variables, entrenar, predecir y evaluar. No es parte del entrenamiento final de PM2.5.

5. Modelo Lasso Regression

Lasso Regression es una regresion lineal con regularizacion L1. Su ventaja es que permite interpretar coeficientes y reducir el peso de variables poco utiles. Como trabaja con operaciones numericas, en este proyecto se entrena solo con variables numericas.

Variables usadas por Lasso:

- PM 10
- SO2
- NO2
- O3
- CO
- ANO
- MES
- DIA
- HORA

Importante: ESTACION no se usa en Lasso porque es texto. Aunque tecnicamente podria codificarse a variables dummy, se dejo fuera para mantener un modelo lineal simple, coherente y facil de explicar.

6. Modelo Decision Tree Regressor

Decision Tree Regressor divide los datos mediante reglas del tipo variable $\leq$ umbral. Cada camino lleva a una hoja donde se calcula una prediccion promedio de PM2.5. Este modelo permite capturar relaciones no lineales y umbrales, por eso funciona mejor en este proyecto.

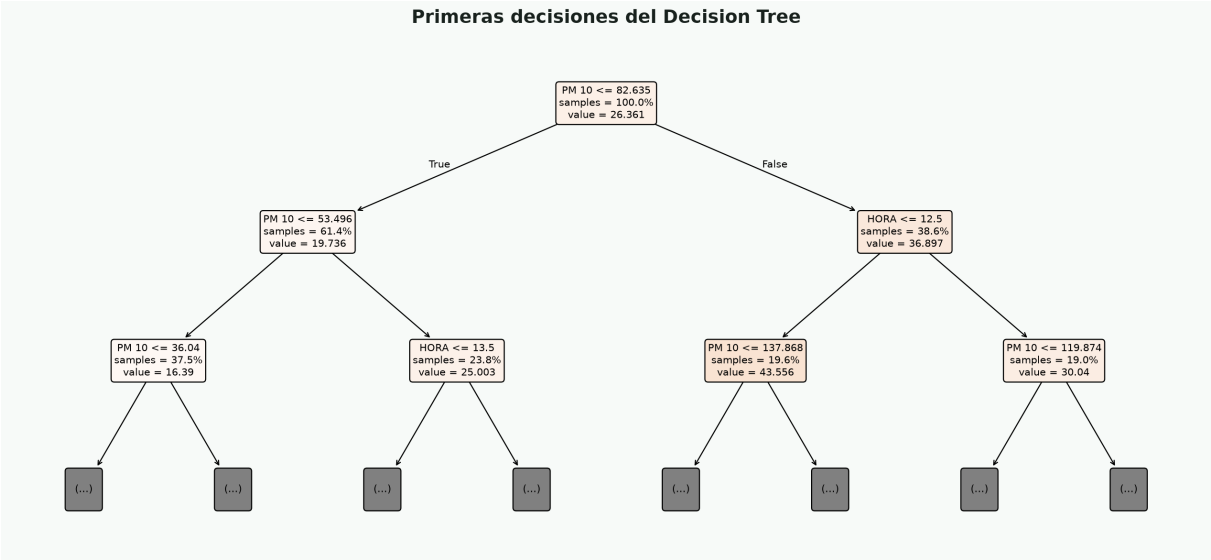
Variables usadas por Decision Tree:

- PM 10
- SO2
- NO2
- O3
- CO
- ANO
- MES
- DIA
- HORA
- ESTACION

La variable ESTACION se transforma internamente con OneHotEncoder. Esto convierte cada estacion en columnas numericas de 0 y 1, por ejemplo ESTACION_ ATE o ESTACION_ SAN BORJA. Asi el arbol puede usar la estacion sin trabajar directamente con texto.

Primera decision del arbol: PM 10 $\leq$ 82.64.

Primeras decisiones del arbol



7. Comparacion de modelos

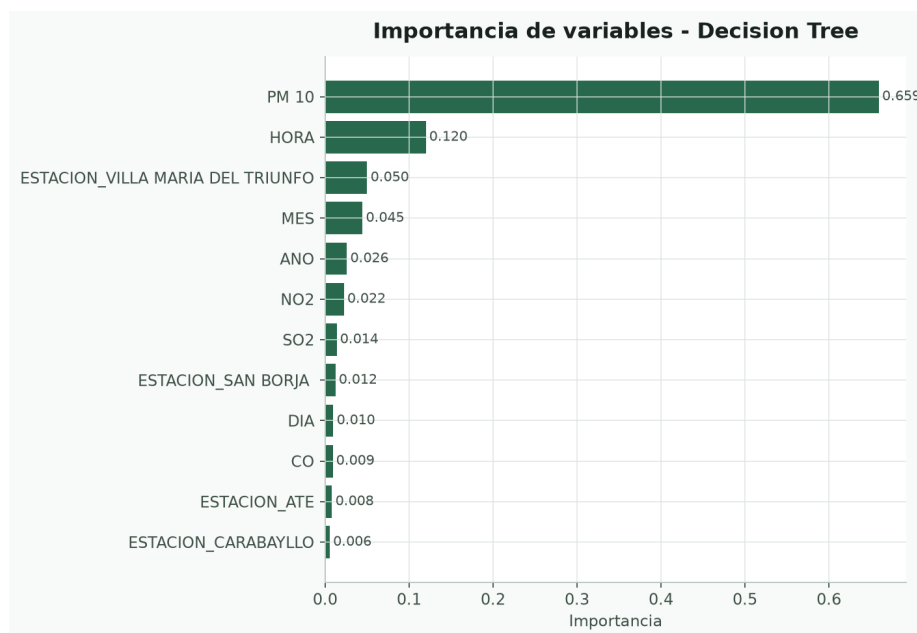
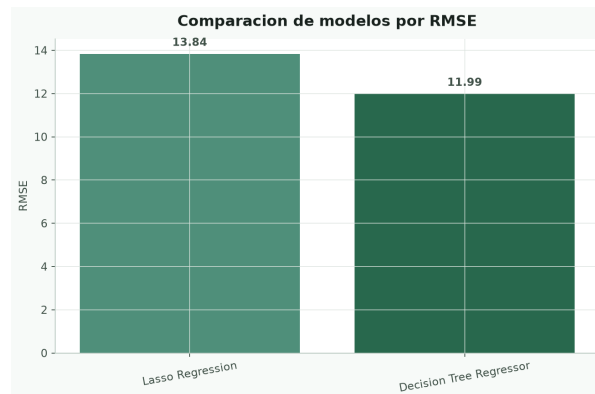
Modelo	MAE	RMSE	R2
Lasso Regression	9.5052	13.8354	0.3773
Decision Tree Regressor	7.9738	11.9852	0.5327

Interpretacion de metricas:

- MAE: error absoluto promedio. Menor es mejor.
- MSE: error cuadratico medio. Penaliza errores grandes.
- RMSE: raiz del MSE. Esta en la misma unidad que PM2.5. Menor es mejor.
- R2: proporcion de variacion explicada por el modelo. Mayor es mejor.

En la ejecucion actual, Decision Tree Regressor obtiene menor MAE, menor RMSE y mayor R2. Esto indica que representa mejor los patrones del dataset, especialmente relaciones por umbrales entre PM10, hora, mes y estacion.

Graficos principales



8. Archivos .pkl y uso en el dashboard

Los .pkl se generan con joblib despues de entrenar los modelos. No son archivos descargados: son los modelos ya entrenados con el dataset limpio. El dashboard los carga para predecir sin volver a entrenar cada vez.

- models/lasso_regression.pkl: pipeline de Lasso con imputacion, escalado y modelo lineal regularizado.
- models/decision_tree_regressor.pkl: pipeline de Decision Tree con imputacion, OneHotEncoder para ESTACION y arbol de decision.
- app/predictor.py: carga los .pkl, convierte el formulario en una fila de datos y devuelve la prediccion.
- app/templates/resultados.html: formulario donde se ingresan los valores para predecir PM2.5.

9. Conclusion

El proyecto queda organizado de forma simple: primero se limpia el dataset sucio, luego se entrenan dos modelos, se guardan los .pkl y finalmente se muestran resultados en un dashboard. Lasso se mantiene como modelo lineal numerico e interpretable; Decision Tree usa tambien la estacion codificada y obtiene mejor rendimiento para este dataset.